



Scene Classification Competition

CSE281 - Introduction to Artificial Intelligence

Submitted To:

Dr. Maryam Nabil

Eng. Mohammed Essam

Eng. Heba Gama

Submitted By:

Mohamed Atef	24P0061
Jana Karara	24P0200
Sarah Osama Hassan	24P0054
Amira Botros Enaam	24P0284
Hannah Hassan Ahmed	24P0089
Noreen Mohamed Tawfik	24P0147
Rawan Amin Mokhtar	24P0148

5 May 2026

1. Introduction

Scene style classification is a fundamental problem in computer vision where the goal is to categorize indoor environments (such as kitchens, bedrooms, offices, and living rooms) based on visual appearance. This task plays an important role in applications such as robotics, smart home systems, image retrieval, and scene understanding in artificial intelligence systems.

In this project, a deep learning-based approach is used to classify indoor scene images into multiple predefined categories. The dataset consists of labeled images distributed across different scene classes, with variations in lighting conditions, object placement, resolution, and visual noise. These challenges make the classification task non-trivial and require robust feature extraction methods.

To address this problem, a convolutional neural network (CNN) architecture is developed. The system is evaluated using a validation split from the training dataset, and performance is measured using accuracy, loss curves, and a confusion matrix. Furthermore, predictions are generated for a hidden test set to produce a final submission file suitable for Kaggle evaluation.

2. Data Analysis & Pre-processing

2.1 Class Mapping and Label Encoding

A separate mapping file is used to ensure consistent class-to-label conversion. This file is parsed to create two dictionaries:

- **class_to_id**: Maps class names → numeric labels

A **controlled mapping dictionary** which ensures:

- consistent label assignment
 - reproducibility across runs
 - compatibility with submission format requirements
- **id_to_class**: Maps numeric labels → class names

This is later used for interpretation and visualization (e.g., confusion matrix labels).

2.2 Data Cleaning and Validation

Before training, the dataset is filtered using a custom validation function:

```
def check_and_count_invalid(path):
```

Inside the function, each image is:

1. **Checked for corruption**
2. **Reopened for format validation**
3. **Filtered by color mode**
 - Grayscale images (L mode)
 - Palette-based images (P mode)
4. **Converted to RGB if valid**

2.2.1 Tracking statistics

The code maintains a global dictionary:

```
stats = {"corrupted": 0, "grayscale_palette": 0, "valid": 0}
```

which is updated inside the function to report dataset quality.

2.3 Class Distribution Analysis

To understand dataset balance, label frequencies are computed using Counter. This step helps identify whether the dataset suffers from class imbalance.

2.4 Train–Validation Split Strategy

The dataset is split into training and validation sets using a stratified split (80/20). This ensures that each class is proportionally represented in both subsets so that the model is unbiased.

- **Training dataset** → includes augmentation
- **Validation dataset** → clean, non-augmented images

2.5 Image Transformations (Pre-processing)

All images are resized and normalized to ensure consistency before being passed into the CNN model.

- **Resize(224x224)**: ensures fixed input size
- **RandomHorizontalFlip(p=0.5)**: 50% chance to flip the image
- **RandomRotation(15)**: handles orientation changes
- **ColorJitter (brightness=0.2, contrast=0.2)**: randomly changes brightness/contrast to help the model handle different lighting conditions in indoor photos

- **Normalize:** This centers the data around zero, which makes the math for training much faster and more stable.

2.6 Data Augmentation (Training Set Only)

To improve generalization and reduce overfitting, multiple augmentation techniques are applied:

- Random horizontal flipping ($p = 0.5$)
- Random rotation (± 15 degrees)
- Color jitter (brightness and contrast variation)
- (Optional enhancement: resizing consistency handled via transforms)

2.7 DataLoader Configuration

Training loader:

- Shuffle enabled → Mixes the images every epoch so the model doesn't memorize the order
- **num_workers=0:** Uses the main process to load data
- Drop last incomplete batch
- Batch size = 32

Validation loader:

- No shuffling
- Deterministic evaluation

3. Methodology: Model Architectures

3.1. StyleCNN (Custom CNN)

3.1.1. Architecture

The baseline model is implemented in the 4-layer convolutional feature extractor:

- Conv1 → 32 filters → Feature extraction
- Conv2 → 64 filters → Mid-level features
- Conv1 → 128 filters → High-level features
- Conv1 → 256 filters → Deep semantic features

3.1.2. Key features

Each convolution is followed by:

Batch Normalization:

- Ensures no single pixel value becomes so large that it overpowers the others
- Stabilizes training
- Speeds up convergence

LeakyReLU Activation:

- Prevents dead neuron problem
- Ensures gradient flow even for negative activations (gets multiplied by 0.1 to prevent "Dead ReLU" where neurons stop learning entirely)

Max Pooling:

- Reduces the image dimensions by half
- It keeps only the strongest signal in each 2x2 area

3.1.3. Global Pooling

Instead of flattening early feature maps, the model uses:

nn.AdaptiveAvgPool2d((7, 7))

- Ensures fixed-size output regardless of input variation
- Reduces parameter explosion
- Improves spatial generalization

3.1.4. Classifier Design

- **Flatten:** Takes the 3D of data and flattens it into one long 1D list
- **Linear:** Fully connected layer (512 neurons)
- **BatchNorm1d:** For stabilization
- **Dropout(0.4):** Randomly ignores 40% of the neurons during every training step to prevent overfitting

3.2. EfficientNet-B3

3.2.1 Architecture

EfficientNet-B3 is built using a combination of:

- **MBConv (Mobile Inverted Bottleneck Convolution) blocks**
- **Depthwise separable convolutions**

- **Squeeze-and-Excitation (SE) modules**
- **Swish (SiLU) activation function**

These components allow the model to learn rich feature representations while maintaining computational efficiency.

3.2.2 Squeeze-and-Excitation (SE) Mechanism

A key component which improves feature representation by explicitly modeling channel-wise relationships.

The process works as follows:

- **Squeeze step:** Global average pooling compresses spatial information into a channel descriptor.
- **Excitation step:** Fully connected layers learn channel importance weights.
- **Recalibration step:** Feature maps are reweighted based on learned importance.

This allows the network to focus more on informative features and suppress less useful ones.

3.2.3 Custom Implementation

Stem Layer

- Initial convolution layer reduces spatial dimensions
- Batch Normalization stabilizes activations
- SiLU activation improves gradient flow

Feature Extraction Block

A simplified inverted residual block was used:

- Pointwise convolution (expansion phase)
- Depthwise convolution (efficient spatial filtering)
- Squeeze-and-Excitation module
- Projection convolution (compression phase)

Classification Head

- Global average pooling reduces feature maps to a vector
- Fully connected layer maps features to class logits

- Dropout reduces overfitting

3.2.4 Transfer Learning Strategy

To improve convergence and leverage learned representations, transfer learning was applied using pretrained weights from ImageNet.

Weight Initialization Strategy

- Pretrained EfficientNet-B3 weights were loaded
- Matching layers between pretrained model and custom model were copied
- Non-matching layers (especially classifier head) were randomly initialized

3.2.5 Freezing Strategy

During early training stages:

- Backbone layers were frozen
- Only the classifier head was trained

3.2.6 Fine-Tuning Phase

After initial convergence:

- All layers were unfrozen
- Lower learning rates were applied to backbone layers
- Higher learning rate was maintained for classifier head

This ensures:

- Stable preservation of pretrained features
- Adaptation to dataset-specific patterns
- Improved generalization

3.2.7 Optimization Improvements

Several advanced optimization techniques were used:

- **AdamW optimizer**
 - Improves generalization through decoupled weight decay
- **Label smoothing (0.1)**

- Prevents overconfident predictions
- Improves generalization to unseen data
- **Cosine Annealing Warm Restarts**
 - Dynamically adjusts learning rate
 - Helps escape local minima
- **Mixed Precision Training (AMP)**
 - Uses FP16 computation for faster training
 - Reduces GPU memory usage
- **Gradient Scaling**
 - Prevents underflow during backpropagation in mixed precision

3.3. DenseNet121

3.3.1 Key Idea: Feature Reuse

DenseNet is based on the principle that:

Each layer receives the concatenated feature maps of all previous layers.

This design has two major advantages:

- Stronger gradient propagation, reducing vanishing gradient problems
- Feature reuse, where early low-level features (edges, textures) are reused in deeper layers

This makes DenseNet particularly effective for fine-grained visual classification tasks, such as distinguishing indoor scene styles.

3.3.2 Architecture Design

- Initial Convolution:
 - 7×7 convolution → 64 filters
 - Followed by BatchNorm + ReLU + MaxPooling

- Dense

Blocks:

The network is composed of four dense blocks:

- Block 1: 6 layers

- Block 2: 12 layers
- Block 3: 24 layers
- Block 4: 16 layers

Each dense layer performs:

- 1×1 convolution (bottleneck layer)
- 3×3 convolution (feature extraction)
- Feature concatenation with previous outputs
- Transition

Layers:

Between dense blocks:

- 1×1 convolution (channel reduction)
- 2×2 average pooling (spatial downsampling)
- Final Layers:
 - Batch normalization
 - Global average pooling
 - Fully connected classifier (17 classes)

3.3.3 Regularization and Stability Improvements

To improve generalization and prevent overfitting, several modifications were introduced:

- Dropout ($p = 0.5$) applied before the classifier
→ reduces co-adaptation between neurons
- Weight Decay (L2 Regularization = $1e-4$)
→ penalizes large weights and encourages simpler decision boundaries
- Batch Normalization throughout the network
→ stabilizes activation distributions and speeds up convergence

3.3.4 Transfer Learning Strategy

The model was initialized using pretrained ImageNet weights from DenseNet-121, with a custom adaptation strategy:

- Pretrained convolutional weights were loaded

- The final classification layer was replaced to match 17 scene classes
- Only compatible layers were transferred (excluding classifier mismatch)

This allowed the model to:

- Retain general visual features (edges, textures, shapes)
- Adapt specifically to indoor scene classification

3.3.5 Training Behavior

Learning rate scheduling was applied using:

- ReduceLROnPlateau, which lowers learning rate when validation loss stagnates

This allowed the model to fine-tune gradually and avoid oscillations near convergence.

3.4. ResNet

3.5.1 Core Idea: Residual Learning

Traditional deep networks attempt to learn a direct mapping:

$$H(x)$$

ResNet instead reformulates this as:

$$F(x) = H(x) - x \Rightarrow H(x) = F(x) + x$$

This means the network learns a **residual function** rather than the full transformation.

The key component enabling this is the **skip connection**, where the input of a layer is directly added to its output.

3.5.2 Why Residual Connections Matter

Residual connections solve two major problems in deep networks:

- **Vanishing Gradient Problem**
→ Gradients flow directly through skip connections, allowing earlier layers to learn effectively
- **Degradation Problem**
→ Increasing depth no longer guarantees worse performance

3.5.3 Architecture Overview

The ResNet-based model used in this project follows a standard pretrained backbone structure:

- Initial convolution layer (7×7 kernel)
- Max pooling layer for spatial reduction
- Residual blocks consisting of:
 - Convolution layers (3×3 filters)
 - Batch Normalization
 - ReLU activation
 - Skip connections (identity mapping)
- Global Average Pooling layer
- Fully connected classification layer (17 output classes)

3.5.4 Transfer Learning Strategy

Key adaptation steps:

- The final fully connected layer was replaced to match the **17 scene classes**
- Earlier convolutional layers were kept intact to preserve general visual feature extraction
- Fine-tuning was applied to adapt the model to indoor scene textures and layouts

4. General Training Strategy

4.1. Head Initialization (Freezing)

- The Feature Extraction backbone (the layers responsible for identifying shapes and textures) was frozen by disabling gradient calculations. Training was restricted exclusively to the Classification Head (the final fully connected layers).
- By freezing the features, we prevented the randomly initialized weights of the classifier from distorting the feature maps during the initial steps of gradient descent.
- This significantly reduces the risk of Feature Distortion, where the initial random weights of a new classifier can ruin the pre-learned patterns in the backbone.

4.2. Global Fine-Tuning (Unfreezing)

- All layers across the entire architecture were unfrozen, allowing the optimizer to update every weight in the model.
- To preserve the stability of the model, a reduced learning rate (typically 10% of the initial rate) was used.
- This phase allows the backbone to "specialize." It fine-tunes the general visual features to better distinguish the subtle nuances specific to the target dataset (e.g., specific architectural details or design textures).

5. Loss Function & Class Imbalance Handling

5.1 Objective Function: Cross-Entropy Loss

- **Mechanism:** This function measures the difference between the model's predicted probability distribution and the actual labels. It penalizes incorrect predictions (the more confident the model is in a wrong answer, the higher the penalty).
- **General Application:** This is the standard choice for multi-class classification tasks as it effectively drives the model toward a "winner-takes-all" probability for the correct category.

5.2 Addressing Dataset Imbalance

In real-world datasets, some categories often have significantly more samples than others. If unaddressed, the model may develop a majority-class bias, simply guessing the most frequent class to achieve a high "fake" accuracy. To mitigate this, the following strategies were integrated:

- **Stratified Splitting:** During the data division phase, Stratified Sampling was used to ensure that the ratio of classes in the training, validation, and test sets is identical to the original dataset. This prevents the model from missing rare classes during any stage of evaluation.
- **Data Augmentation (Class-Specific):** Transformation techniques (flips, rotations, and color jitters) were applied more heavily to minority classes. This artificially increases the variety of samples available for the model to learn from, effectively balancing the "signal" received from each category.
- **Weighted Loss:**

$$w_c = \frac{1}{\log(1 + n_c)}$$

This forces the optimizer to treat every category as equally important, regardless of its frequency in the folder structure.

6. Optimizer & Learning Strategy

6.1 Optimization Algorithm: Adaptive Learning

The model utilizes the **Adam (Adaptive Moment Estimation)** optimizer.

- **Mechanism:** It computes individual adaptive learning rates for different parameters. It uses estimates of the first and second moments of the gradients to adapt the update rule.
- **Weight Decay:** A L2 Regularization (Weight Decay) was applied within the optimizer. This penalizes excessively large weights, forcing the model to learn simpler, more generalizable patterns rather than memorizing noise in the training data.

6.2 Learning Rate Scheduling

To ensure the model settles into the global minimum of the loss surface without overshooting, a dynamic scheduling strategy was used:

- **ReduceLROnPlateau Scheduler:** The learning rate is monitored against the Validation Loss.
- **Decay Mechanism:** If the validation loss fails to improve for a set number of epochs (patience), the scheduler automatically reduces the learning rate by a specific factor (e.g., 0.1).
- **Objective:** This allows the model to take large "steps" early in training to find the general region of the optimal solution, and smaller, more precise "steps" later to fine-tune the weights.

6.3 Convergence Monitoring (Early Stopping)

To maximize efficiency and prevent **Overfitting**, an **Early Stopping** mechanism was implemented.

- **Termination Criteria:** The training process is terminated if the validation performance does not improve for a predefined number of consecutive epochs.
- **General Benefit:** This ensures that the model does not continue training to the point where it begins to memorize the training set, effectively "stopping while the model is at its best."

7. Training Procedure

Training setup:

- Epochs: 30
- Batch size: 32

- GPU acceleration enabled (if available)

Process flow:

For each epoch:

- Forward pass
- Compute loss
- Backpropagation
- Optimizer step
- Validation
- Save best model

Stability techniques:

- Gradient clipping (implicit best practice)
- Early stopping
- Dropout regularization

8. Challenges & Optimization

8.1 Vanishing and Exploding Gradients

In deep architectures, as gradients are backpropagated through many layers, they can either shrink to zero (Vanishing) or grow exponentially (Exploding). This makes the early layers of a network nearly impossible to train.

- **Optimization: Batch Normalization** was implemented after convolutional and linear layers. By normalizing the activations of each batch to have a mean of zero and unit variance, we ensure that the signal remains within a range where the activation functions (like LeakyReLU) remain sensitive, stabilizing the training process.

8.2 Overfitting and Generalization Gap

A primary challenge in scene classification is **Overfitting**, where a model learns the specific noise or background details of the training images instead of the actual style features.

- **Optimization:** A multi-layered defense was used:
 - **Dropout:** Randomly deactivating neurons during training to prevent "co-adaptation" (where neurons rely too heavily on each other).

- **Data Augmentation:** Artificially expanding the dataset using geometric and color transformations. This forces the model to learn that a "Bohemian" room is still Bohemian even if it is flipped, rotated, or seen in different lighting.
- **Weight Decay:** Adding a penalty to the loss function for large weights to keep the model's internal math "simple."

8.3 Computational Constraints & Image Resolution

High-resolution images contain more information but require exponentially more VRAM and processing time.

- **Optimization: Adaptive Pooling** was utilized. This allowed the model to accept various input scales while maintaining a fixed-size feature map for the classification head. Additionally, resizing images to a standard 224x224 resolution provided a balance between retaining architectural detail and maintaining high training throughput (Batch Size).

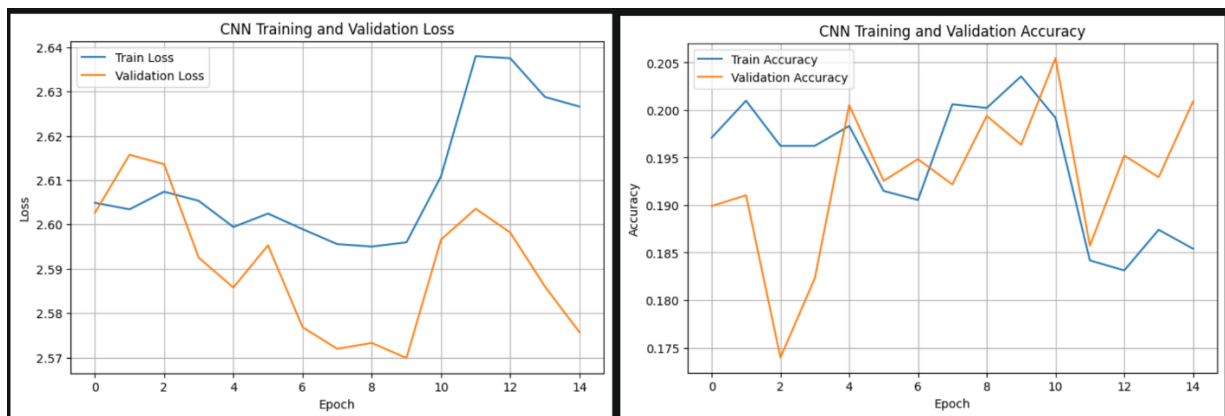
8.4 Feature Spatial Loss

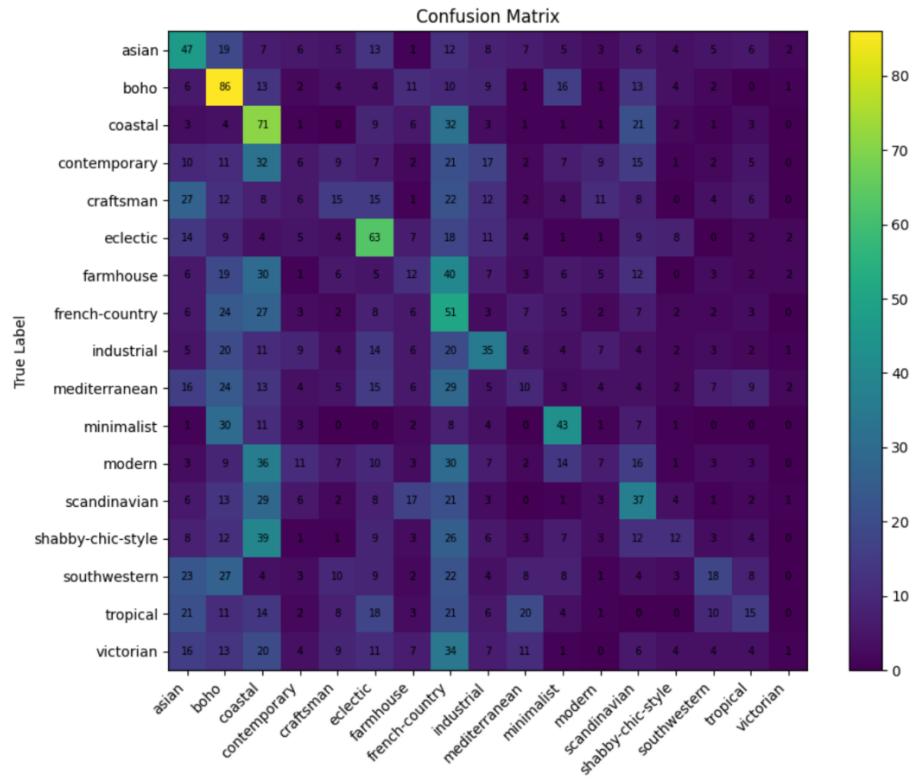
Standard pooling layers (like MaxPool) can sometimes discard important spatial information.

- **Optimization:** The use of **LeakyReLU** instead of standard ReLU ensures that even when a neuron is "inactive" (negative value), a small gradient still flows. This prevents "Dead Neurons" and ensures that every part of the feature extractor contributes to the final style prediction.

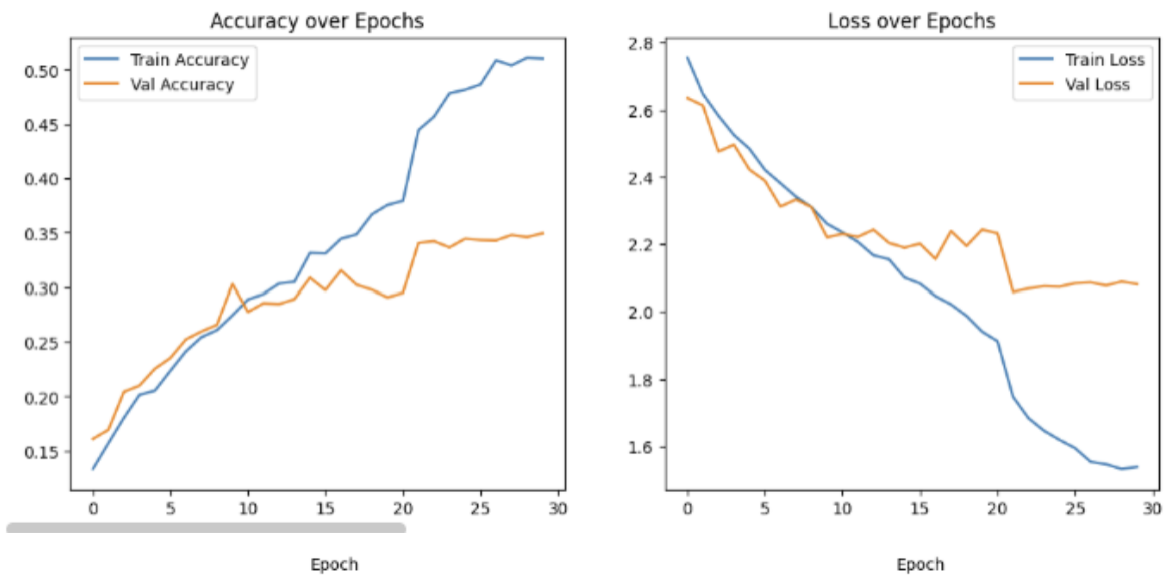
9. Visualization

9.1 EfficientNet – B3

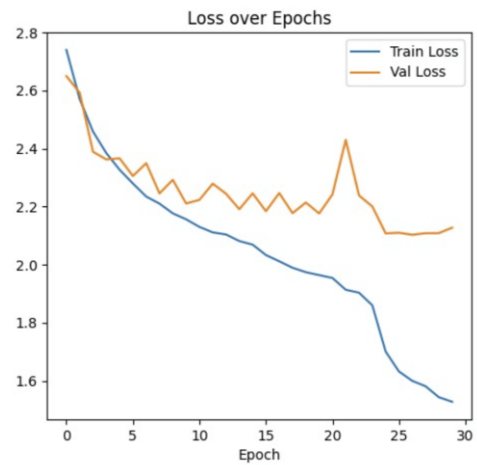
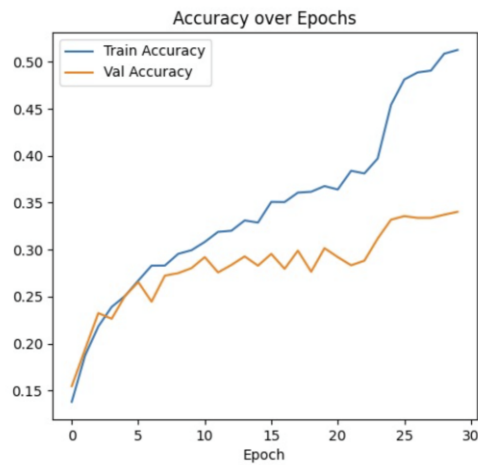




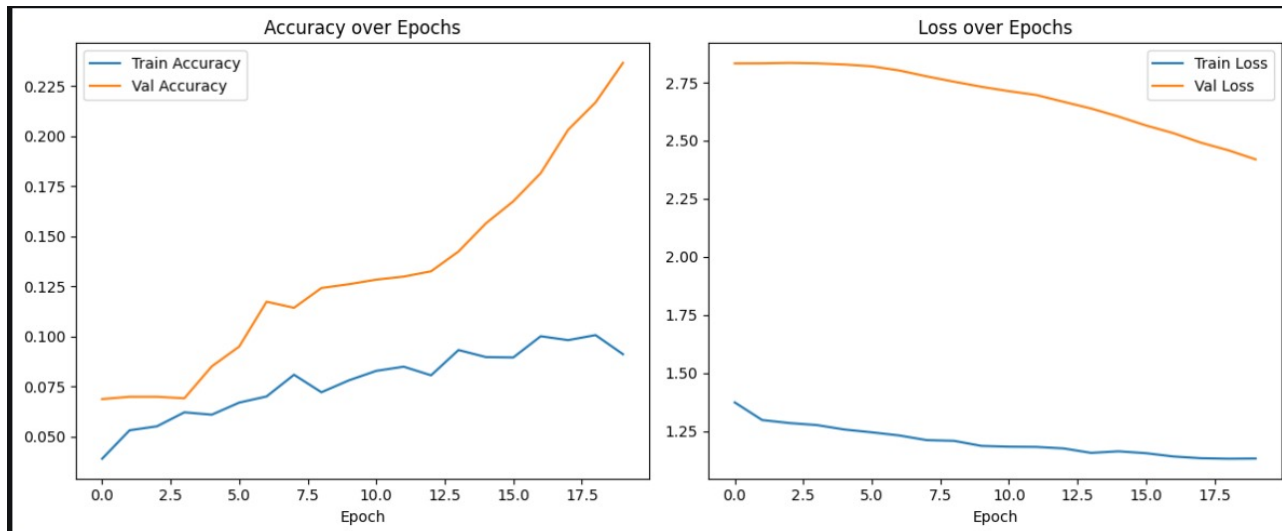
9.2 DenseNet121

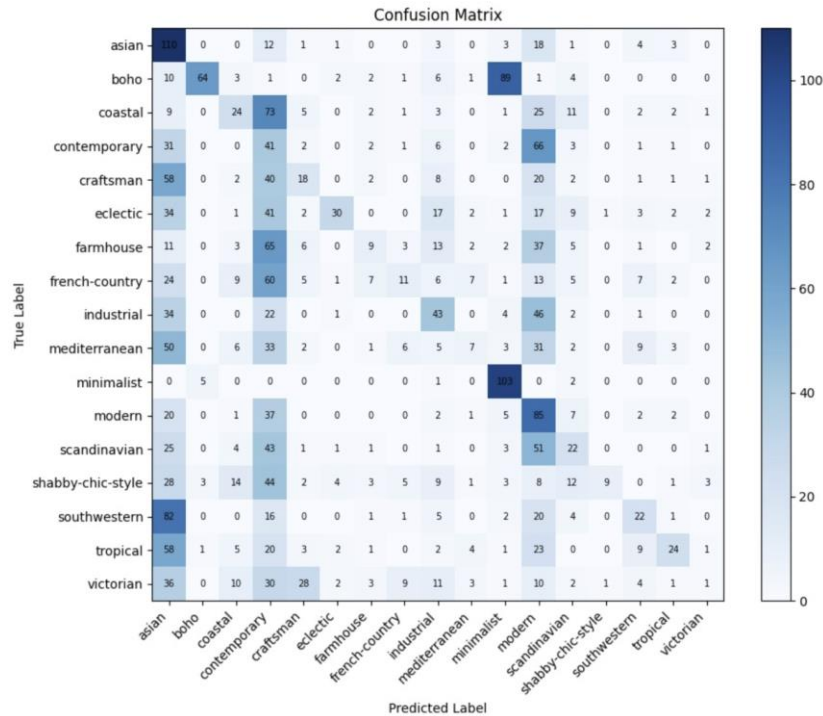


9.3 StyleCNN



9.4 ResNet





10. Experimental Results

Model	Best Validation Accuracy (%)	Epochs	Notes
StyleCNN	34.03%	30	Baseline model with higher variance
ResNet	23.66%	20	Stable gradients and reliable convergence
DenseNet	34.94%	30	Stable training with strong feature reuse
EfficientNet	20.55%	15	Strong generalization and efficient computation

11. Analysis & Discussion

Why Accuracy Started Low

At the beginning of training, the classifier layers were randomly initialized while the backbone networks contained pretrained ImageNet features. Since the classifier initially had no understanding of the indoor scene categories, early predictions were highly inaccurate, resulting in low validation accuracy and high loss values.

Effect of Data Augmentation

Data augmentation significantly improved generalization performance by exposing the models to multiple visual variations of the same scene:

- Rotations improved orientation robustness
- Horizontal flipping reduced positional bias
- Color jitter improved lighting invariance
- Random cropping enhanced spatial robustness

This prevented the models from memorizing fixed image patterns and improved validation accuracy.

Effect of Unfreezing Layers

Initially freezing the backbone preserved pretrained feature representations and stabilized early optimization. After convergence of the classifier head, global fine-tuning allowed deeper layers to specialize for indoor scene textures and layouts. This improved:

- Feature adaptation
- Validation accuracy
- Scene-specific discrimination

Effect of Weighted Loss

Weighted loss reduced majority-class bias by assigning higher penalties to underrepresented categories. This improved recognition of rare scene styles and produced more balanced classification performance across all classes.

Overfitting and Underfitting Observations

Several models initially showed signs of overfitting:

- Training accuracy increased rapidly
- Validation accuracy plateaued early

This behavior was mitigated using:

- Dropout
- Weight decay
- Data augmentation

- Learning rate scheduling

Underfitting was mainly observed during the early frozen-backbone phase when only the classifier head was trainable.

Limitations

Despite the improvements, several limitations remained:

- Moderate overall validation accuracy due to high visual similarity between some classes
- DenseNet introduced higher computational and memory costs because of dense feature concatenation
- Dataset imbalance still affected minority-class prediction quality
- Some scene styles shared highly overlapping textures, colors, and furniture layouts, making fine-grained discrimination difficult

12. Conclusion

This project explored multiple deep learning architectures for indoor scene style classification, including StyleCNN, EfficientNet-B3, DenseNet121, and ResNet.

Among the tested architectures, the transfer learning–based models significantly outperformed the baseline StyleCNN due to their ability to leverage pretrained ImageNet features. EfficientNet-B3 demonstrated strong computational efficiency and balanced performance, DenseNet121 provided stable feature reuse and strong gradient propagation, while ResNet achieved reliable convergence through residual learning.

The experiments highlighted the importance of:

- Transfer learning
- Data augmentation
- Fine-tuning strategies
- Regularization techniques
- Learning rate scheduling

for achieving stable and generalizable performance in computer vision tasks.

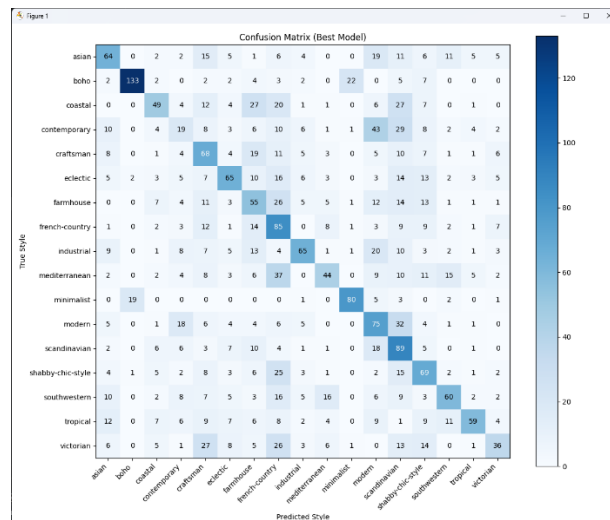
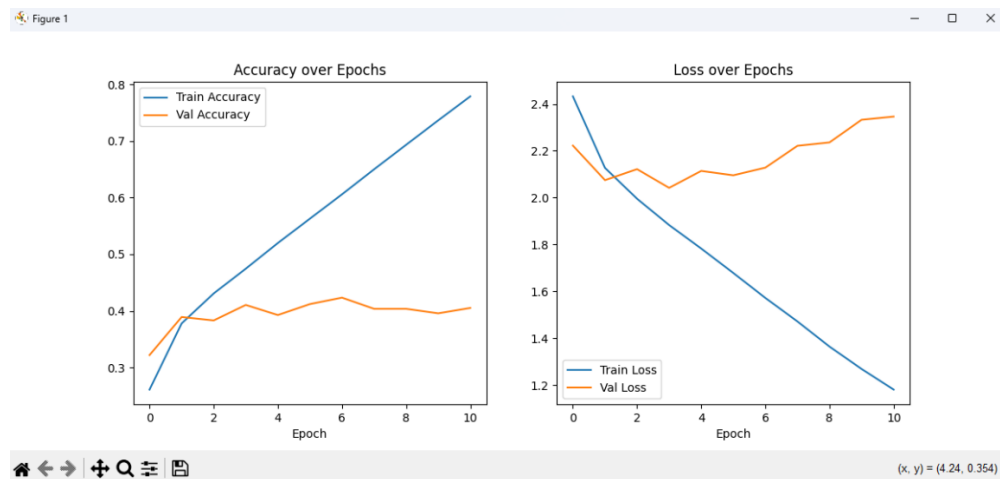
Overall, the project demonstrated that modern convolutional neural networks can effectively learn complex indoor scene representations despite challenges such as class imbalance, overlapping visual styles, and dataset variability.

13. Bonus Material

13.1 ViT

Best Validation Accuracy:

42.35%

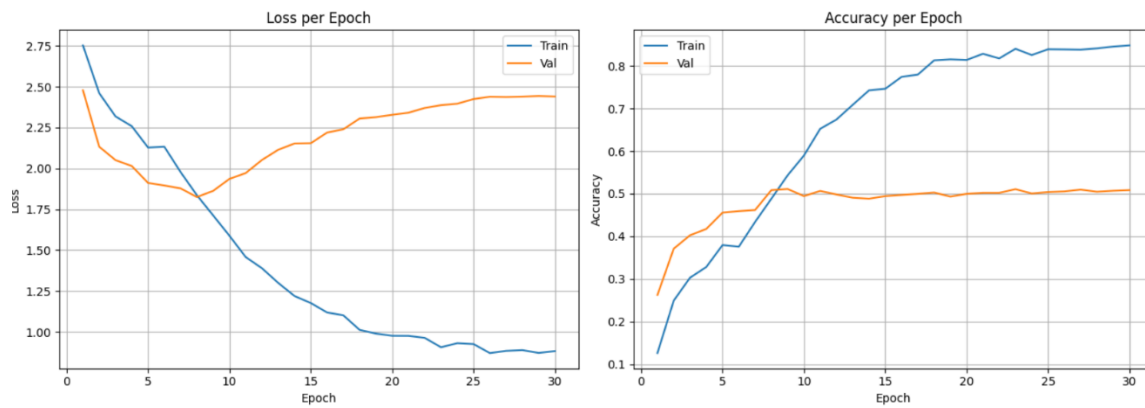


By bypassing standard library wrappers, the pipeline performs a surgical **weight transplantation**, mapping pre-trained feature tensors onto a custom-built Transformer backbone. This enables the model to utilize high-level feature extraction while adapting to specific interior design nuances through a bespoke classification head.

13.2 ConvNext

Best Validation Accuracy:

51.12%



ConvNeXt adopts the macro-design of Transformers (like larger kernels and fewer activation functions) but keeps the efficiency and simplicity of standard convolutions. By utilizing the **Large** variant, our model is deep enough to distinguish between highly nuanced interior styles that smaller models often miss.